Guía de comandos básicos de Git y Bash

Referencia rápida: comando y para qué sirve

Bash (terminal)

Navegación

Comando	Para qué sirve
`pwd`	Muestra en qué carpeta estás
`ls`	Lista archivos y carpetas
`ls -la`	Lista todo, incluso ocultos (como .git), con detalles
`cd carpeta`	Entra a una carpeta
`cd ..`	Sube una carpeta
`cd ~`	Va a tu carpeta de usuario
`cd -`	Regresa a la carpeta anterior

Archivos y carpetas

Comando	Para qué sirve
`mkdir nombre`	Crea una carpeta
`touch archivo.txt`	Crea un archivo vacío
`cp origen destino`	Copia un archivo
`cp -r carpeta destino`	Copia una carpeta completa
`mv origen destino`	Mueve o renombra un archivo o carpeta
`rm archivo`	Borra un archivo (sin papelera, no se recupera)
`rm -rf carpeta`	Borra una carpeta y todo su contenido (con mucho cuidado)

Ver y buscar contenido

Comando	Para qué sirve
`cat archivo`	Muestra el contenido de un archivo
`head -n 10 archivo`	Muestra las primeras 10 líneas
`tail -n 10 archivo`	Muestra las últimas 10 líneas
`echo "texto"`	Imprime texto
`echo "texto" > archivo`	Escribe texto en un archivo (sobrescribe)
`echo "texto" >> archivo`	Agrega texto al final del archivo
`grep "palabra" archivo`	Busca texto dentro de un archivo
`find . -name "*.py"`	Busca archivos por nombre

Utilidades

Comando	Para qué sirve
`clear (o Ctrl+L)`	Limpia la pantalla
`history`	Muestra los comandos que has usado
`man comando / comando --help`	Muestra la ayuda de un comando
`Ctrl+C`	Cancela el comando en ejecución
`Tab`	Autocompleta nombres de archivos y comandos
`Flecha arriba / abajo`	Recorre los comandos anteriores

Git

Configuración inicial (una sola vez)

Comando	Para qué sirve
`git config --global user.name "Tu Nombre"`	Define tu nombre para los commits
`git config --global user.email "correo@ejemplo.com"`	Define tu correo para los commits
`git config --global init.defaultBranch main`	Hace que las ramas nuevas se llamen main

Crear o descargar un repo

Comando	Para qué sirve
`git init`	Convierte la carpeta actual en un repo de Git
`git clone URL`	Descarga un repo de GitHub a tu computadora

Ciclo diario de trabajo

Comando	Para qué sirve
`git status`	Muestra qué archivos cambiaron y cuáles están listos para commit
`git add archivo`	Prepara un archivo para el commit
`git add .`	Prepara todos los cambios
`git commit -m "mensaje"`	Guarda los cambios preparados con un mensaje
`git push origin main`	Sube tus commits a GitHub
`git pull origin main`	Descarga e integra los cambios de GitHub

Ver historial y diferencias

Comando	Para qué sirve
`git log --oneline`	Historial resumido, un commit por línea
`git log --oneline --graph --all`	Historial con forma de árbol de ramas
`git diff`	Muestra los cambios que aún no preparaste
`git diff --staged`	Muestra los cambios preparados para commit
`git show`	Muestra los detalles del último commit

Ramas

Comando	Para qué sirve
`git branch`	Lista las ramas
`git branch nueva`	Crea una rama
`git switch nombre`	Cambia de rama
`git switch -c nueva`	Crea una rama y se cambia a ella
`git merge rama`	Une rama con la rama en la que estás
`git branch -d rama`	Borra una rama ya unida
`git branch -M main`	Renombra la rama actual a main

Remotos (GitHub)

Comando	Para qué sirve
`git remote -v`	Muestra a qué URL está conectado tu repo
`git remote add origin URL`	Conecta tu repo local con uno de GitHub
`git remote set-url origin URL`	Cambia la URL del remoto
`git fetch`	Descarga los cambios de GitHub sin integrarlos
`git push -u origin main`	Sube y enlaza main, para que después baste git push

Deshacer cosas

Comando	Para qué sirve
`git restore archivo`	Descarta los cambios no guardados de un archivo
`git restore --staged archivo`	Saca un archivo de los preparados, sin perder los cambios
`git commit --amend -m "nuevo mensaje"`	Corrige el último commit (si aún no lo subiste)
`git revert ID_COMMIT`	Crea un commit nuevo que deshace otro, de forma segura
`git reset --hard`	Descarta todos los cambios locales (destructivo)
`git merge --abort`	Cancela un merge a medias

Otros útiles

Comando	Para qué sirve
`git stash`	Guarda cambios temporalmente sin hacer commit
`git stash pop`	Recupera los cambios guardados con stash
`.gitignore (archivo)`	Lista archivos y carpetas que Git debe ignorar (contraseñas, node_modules, etc.)

Flujo típico

```
git status
git add .
git commit -m "Describe lo que hiciste"
git pull origin main
git push origin main
```

Haz **pull** antes de **push** para evitar el error de "fetch first".